

DEEP BLUFFING

AYUSH MISHRA

Roll Number: 250253

Contents

1	Introduction and Project Overview	3
1.1	System Architecture and Modules	3
2	Software Architecture and Component Design	4
2.1	Implemented Modules	4
2.2	UML Class Diagram	4
2.3	The Deterministic Fixed-Limit Betting State Machine	6
3	Combinatorial Hand Evaluation Logic	8
3.1	Overall Poker Game Loop Flow	9
4	Artificial Intelligence Module via Monte Carlo Rollouts	12
4.1	Mathematical Framework	12
4.2	Decision Policy Mapping	14
5	Challenges Faced and System Limitations	14
6	Future Extensions and Structural Enhancements	15

List of Figures

1	Overall Software Component Architecture and Dependency Mappings. . .	4
2	UML Class Interactions and Core Attributes Mapping.	5
3	Control Flowchart of the Deterministic Betting Machine.	7
4	Sequential Operational Flow of the Poker Game Controller.	10
5	Process Flow of the Monte Carlo Simulation Agent.	13

List of Tables

1	Heads-Up Limit Texas Hold'em Betting Structure.	6
2	Poker Hand Rankings with Evaluation Priority.	8

1 Introduction and Project Overview

Imperfect-information games represent a primary frontier in artificial intelligence research. Unlike perfect-information games such as Chess or Go, where the entire state of the environment is visible to all participants, poker forces an agent to execute decisions under severe environmental uncertainty and operational opacity.

This technical report details the implementation of a high-performance Heads-Up Limit Texas Hold'em Poker Engine paired with an Artificial Intelligence decision agent based on Monte Carlo Simulation. Built following robust object-oriented software design paradigms, the system models poker rules deterministically while exposing interfaces for programmatic simulation, benchmarking, and interactive Human vs. AI gameplay.

1.1 System Architecture and Modules

The project codebase is partitioned into distinct modular layers to decouple core rule evaluation from state machine execution and heuristic agent logic. The directory tree below illustrates the exact structural layout of the implemented system:

```
1 project/
2 |-- engine/
3 |   |-- card.py
4 |   |-- deck.py
5 |   |-- hand.py
6 |   |-- evaluator.py
7 |   |-- player.py
8 |   |-- betting.py
9 |   |-- game.py
10 |   `-- human_player.py
11 |-- ai/
12 |   |-- montecarlo.py
13 |   |-- policy.py
14 |   `-- opponent.py
15 |-- utils/
16 |   |-- constants.py
17 |   `-- enums.py
18 |-- agent.py
19 |-- benchmark.py
20 |-- human_vs_ai.py
21 |-- main.py
22 `-- README.md
```

Listing 1: Directory Layout of the Completed Poker System

2 Software Architecture and Component Design

To sustain clean state mutations across complex game loops, the application utilizes a Model-Controller-view architecture. Figure 1 provides an overview of the system dependencies.

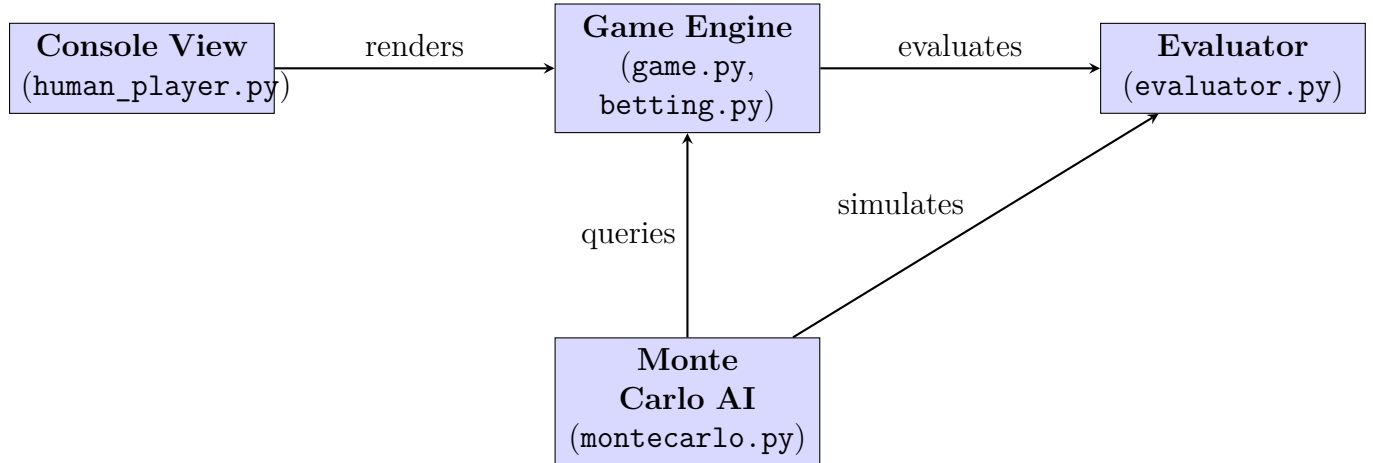


Figure 1: Overall Software Component Architecture and Dependency Mappings.

2.1 Implemented Modules

Each class handles a specific responsibility to keep internal state transitions completely deterministic:

- **Card Class:** Represents an individual playing card. Stores integer rank and suit indices to avoid text parsing overhead during comparative logical executions.
- **Deck Class:** Holds 52 unique Card configurations. Implements clean routines for shuffling (Fisher-Yates protocol), dealing, burning cards, and full-deck resets.
- **Hand Class:** Tracks the private pocket cards belonging to each active player.
- **Player Class:** Tracks player-specific telemetry, containing variables for chip counts, current stakes contributed to the active pot, folded states, all-in thresholds, and hole cards.

2.2 UML Class Diagram

Figure 2 depicts the structural relationship and inheritance boundaries between the core engine entities.

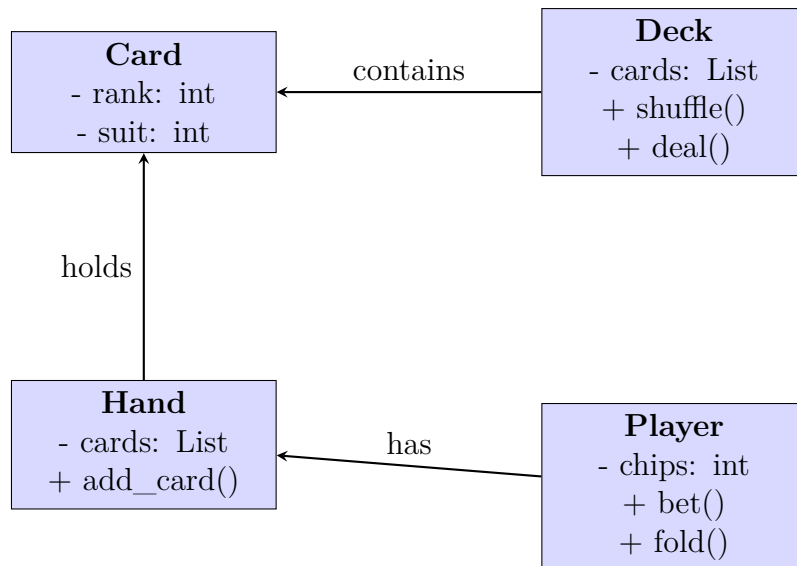


Figure 2: UML Class Interactions and Core Attributes Mapping.

2.3 The Deterministic Fixed-Limit Betting State Machine

The `BettingRound` module enforces a strict state machine to handle chip management and turn order. Table 1 displays the configurations for Heads-Up Limit Texas Hold'em.

Table 1: Heads-Up Limit Texas Hold'em Betting Structure.

Betting Stage	Fixed Bet Unit	Initial Bet	Maximum Raises	Action Limit
Pre-Flop	Small Bet	1 Big Blind	3	4 Bets Max
Flop	Small Bet	0 / 1 Small Bet	3	4 Bets Max
Turn	Big Bet	0 / 1 Big Bet	3	4 Bets Max
River	Big Bet	0 / 1 Big Bet	3	4 Bets Max

Figure 3 maps out how player inputs shift action boundaries, handle folds, resolve checks, and enforce the three-raise limit (or raise cap).

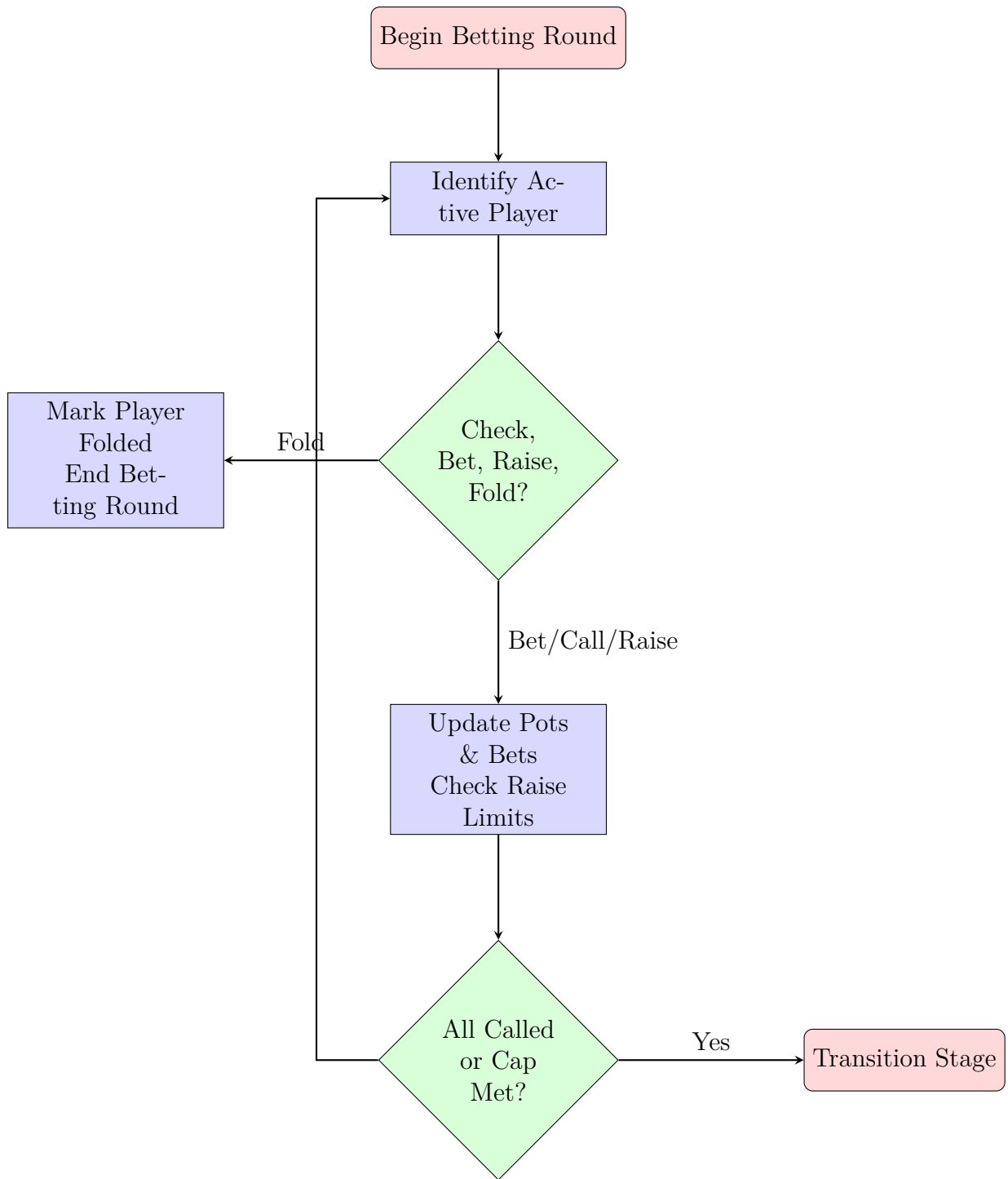


Figure 3: Control Flowchart of the Deterministic Betting Machine.

3 Combinatorial Hand Evaluation Logic

The hand ranking system uses a complete evaluation of all five-card combinations. In any given round of Texas Hold'em, a player's final hand strength is calculated from their 2 private hole cards and the 5 community cards. This requires extracting the best 5-card combination from the total set of 7 cards:

$$\binom{7}{5} = 21 \text{ combinations}$$

Table 2: Poker Hand Rankings with Evaluation Priority.

Rank	Score	Category Name	Primary Comparison Factor	Tie-breaker (Kickers)
	10	Royal Flush	High Card (Fixed Ace)	None
	9	Straight Flush	Top Card Value	None
	8	Four of a Kind	Quad Value	Single Card Value
	7	Full House	Three-of-a-Kind Value	Pair Value
	6	Flush	Highest Card in Suit	Secondary Cards Descending
	5	Straight	Top Card Value	None
	4	Three of a Kind	Set Value	Remaining 2 Cards Descending
	3	Two Pair	High Pair Value	Low Pair, Single Kicker
	2	One Pair	Pair Value	Remaining 3 Cards Descending
	1	High Card	Highest Card Value	Remaining 4 Cards Descending

The evaluation logic converts each five-card hand into a comparative numeric tuple:

$$V_{\text{hand}} = (R_{\text{category}}, X_1, X_2, X_3, X_4, X_5)$$

The program compares these tuples lexicographically. This simplifies tiebreakers by letting the engine run direct, element-wise array comparisons.

Algorithm 1 Combinatorial Hand Evaluation Protocol

```

1: procedure EVALUATEBESTHAND(pocket_cards, community_cards)
2:   seven_cards  $\leftarrow$  pocket_cards  $\cup$  community_cards
3:   combinations_list  $\leftarrow$  all combinations of size 5 from seven_cards
4:   best_score  $\leftarrow$   $(-\infty, 0, 0, 0, 0, 0)$ 
5:   for each combo in combinations_list do
6:     score  $\leftarrow$  SCOREFIVECARDS(combo)
7:     if score > best_score then
8:       best_score  $\leftarrow$  score
9:     end if
10:  end for
11:  return best_score
12: end procedure

```

3.1 Overall Poker Game Loop Flow

The primary controller loop of `PokerGame` executes sequential game state changes. Figure 4 traces these state transitions.

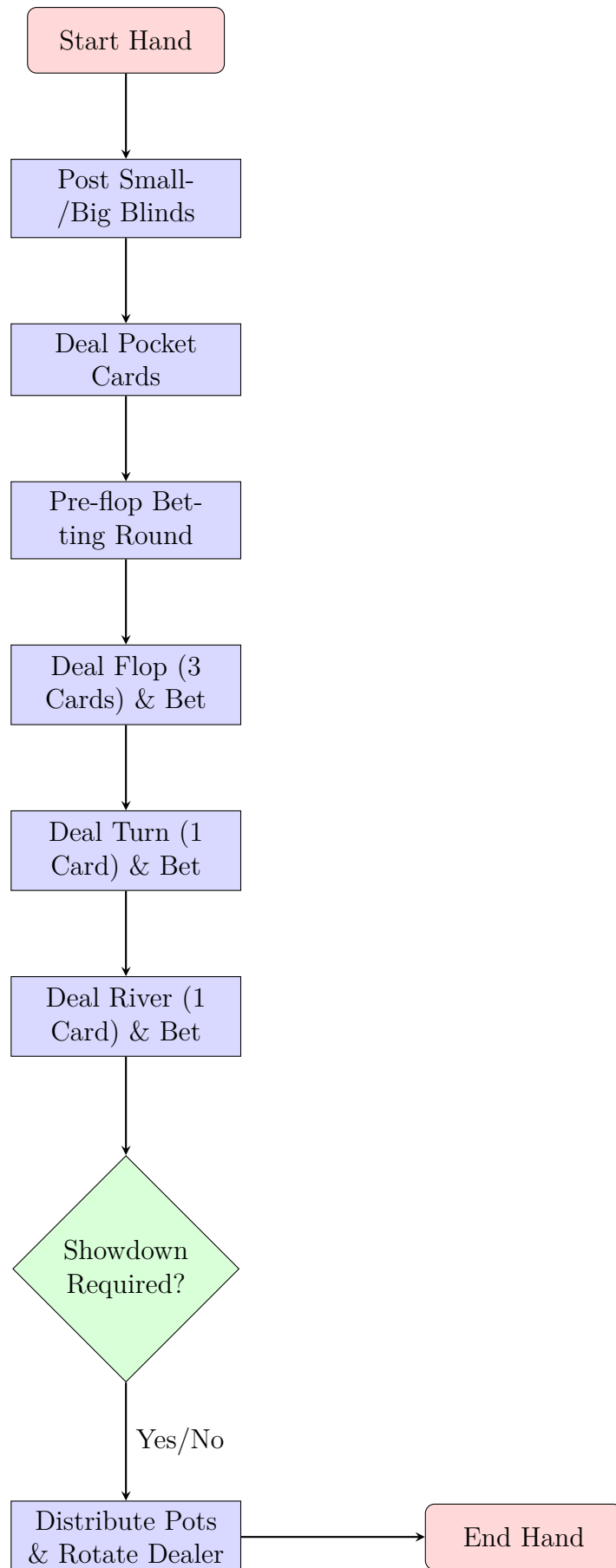


Figure 4: Sequential Operational Flow of the Poker Game Controller.

Algorithm 2 Poker Engine Master Game Controller Loop

```
1: procedure PLAYPOKERHAND(player1, player2)
2:   ROTATEDEALER
3:   POSTBLINDS
4:   DEALPRIVATECARDS
5:   EXECUTEBETTINGROUND(Pre-Flop)
6:   if not ISROUNDACTIVE then
7:     payout winner and exit
8:   end if
9:   DEALFLOP
10:  EXECUTEBETTINGROUND(Flop)
11:  if not ISROUNDACTIVE then
12:    payout winner and exit
13:  end if
14:  DEALTURN
15:  EXECUTEBETTINGROUND(Turn)
16:  if not ISROUNDACTIVE then
17:    payout winner and exit
18:  end if
19:  DEALRIVER
20:  EXECUTEBETTINGROUND(River)
21:  if not ISROUNDACTIVE then
22:    payout winner and exit
23:  end if
24:  EVALUATESHOWDOWN
25:  DISTRIBUTEPOTS
26: end procedure
```

4 Artificial Intelligence Module via Monte Carlo Roll-outs

In imperfect-information environments, computing analytical probabilities across possible opponent cards is computationally intensive. The Monte Carlo approach handles this by sampling cards directly from the remaining deck.

4.1 Mathematical Framework

Let $\mathcal{K}$ represent the set of cards known to the AI agent: its private pocket cards H_{AI} and any visible community cards C_{comm} . The unobserved cards remaining in the deck form a set:

$$\mathcal{U} = \mathcal{D} \setminus \mathcal{K}$$

During each simulation run i , the agent randomly samples a possible opponent hand $H_{\text{opp}}^{(i)}$ from $\mathcal{U}$, and then samples any remaining community cards $C_{\text{rem}}^{(i)}$ needed to complete the 5-card board. The simulated community board is completed as:

$$C_{\text{total}}^{(i)} = C_{\text{comm}} \cup C_{\text{rem}}^{(i)}$$

The outcome variable W_i for each simulation is evaluated as:

$$W_i = \begin{cases} 1.0 & \text{if } \text{Score}(H_{\text{AI}} \cup C_{\text{total}}^{(i)}) > \text{Score}(H_{\text{opp}}^{(i)} \cup C_{\text{total}}^{(i)}) \\ 0.5 & \text{if } \text{Score}(H_{\text{AI}} \cup C_{\text{total}}^{(i)}) = \text{Score}(H_{\text{opp}}^{(i)} \cup C_{\text{total}}^{(i)}) \\ 0.0 & \text{if } \text{Score}(H_{\text{AI}} \cup C_{\text{total}}^{(i)}) < \text{Score}(H_{\text{opp}}^{(i)} \cup C_{\text{total}}^{(i)}) \end{cases}$$

By repeating this sampling process $N = 500$ times, the agent computes an estimated win probability $\hat{P}_{\text{win}}$ using:

$$\hat{P}_{\text{win}} = \frac{1}{N} \sum_{i=1}^N W_i$$

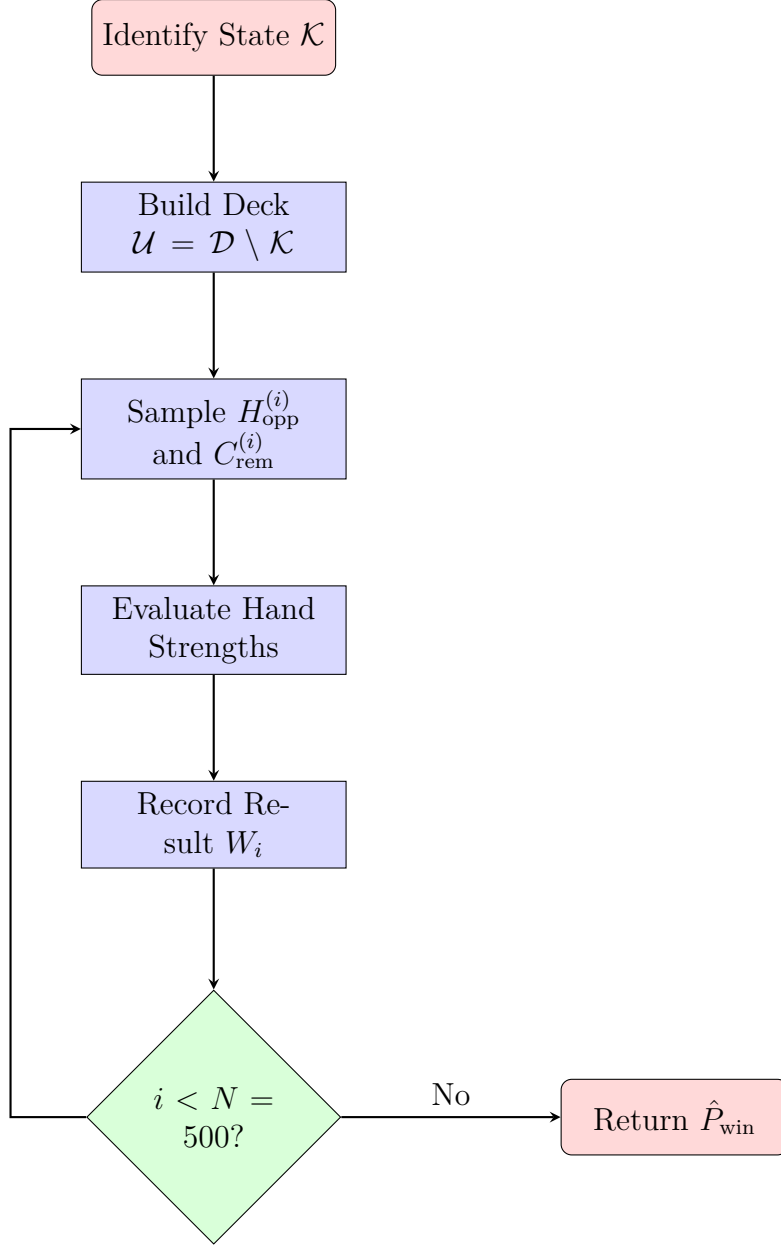


Figure 5: Process Flow of the Monte Carlo Simulation Agent.

4.2 Decision Policy Mapping

The agent uses a threshold-based policy to translate the estimated win probability $\hat{P}_{\text{win}}$ into an game action, balancing aggression and pot management.

Algorithm 3 Monte Carlo Simulation Algorithm

```
1: procedure RUNMONTECARLOSIMULATION(pocket_cards, community_cards, N)
2:   known_cards  $\leftarrow$  pocket_cards  $\cup$  community_cards
3:   total_wins  $\leftarrow$  0.0
4:   for i = 1 to N do
5:     remaining_deck  $\leftarrow$  GenerateDeck()  $\setminus$  known_cards
6:     opp_pocket  $\leftarrow$  DrawRandomCards(remaining_deck, 2)
7:     rem_comm  $\leftarrow$  DrawRandomCards(remaining_deck, 5 -
      |community_cards|)
8:     sim_community  $\leftarrow$  community_cards  $\cup$  rem_comm
9:     ai_score  $\leftarrow$  EVALUATEBESTHAND(pocket_cards, sim_community)
10:    opp_score  $\leftarrow$  EVALUATEBESTHAND(opp_pocket, sim_community)
11:    if ai_score > opp_score then
12:      total_wins  $\leftarrow$  total_wins + 1.0
13:    else if ai_score == opp_score then
14:      total_wins  $\leftarrow$  total_wins + 0.5
15:    end if
16:  end for
17:  return total_wins / N
18: end procedure
```

Algorithm 4 Threshold-Based Action Policy

```
1: procedure SELECTACTION( $\hat{P}_{\text{win}}$ , call_amount)
2:   if  $\hat{P}_{\text{win}} \geq 0.85$  then
3:     return Action::Raise
4:   else if  $\hat{P}_{\text{win}} \geq 0.65$  then
5:     return RandomChoice(Action::Raise [p=0.4], Action::Call [p=0.6])
6:   else if  $\hat{P}_{\text{win}} \geq 0.45$  then
7:     return if call_amount > 0 then Action::Call else Action::Check
8:   else
9:     return if call_amount > 0 then Action::Fold else Action::Check
10:  end if
11: end procedure
```

5 Challenges Faced and System Limitations

Key developmental challenges during implementation included:

1. **Betting State Synchronization:** Implementing fixed-limit raise constraints while keeping pot calculations synchronized during edge-case folds and early showdown actions.

2. **Opponent Modeling Constraints:** The agent currently models opponent distributions as uniform. This makes the policy vulnerable to players who deviate significantly from average betting habits.
3. **Long-Run Stability Issues:** During automated benchmark runs exceeding 5,000 consecutive hands, rare state synchronization issues occurred in the internal betting round check loops. This intermittent issue is currently under active investigation.

6 Future Extensions and Structural Enhancements

Future development iterations aim to transition the engine from predictive sampling heuristics to formal game-theoretic optimal strategies. Implementing Counterfactual Regret Minimization (CFR) will allow the system to compute complete Nash Equilibrium profiles for heads-up abstractions, eliminating the need for uniform distribution assumptions.

Additionally, integrating deep reinforcement learning architectures (e.g., Deep Q-Networks optimized with self-play pipelines) will enable the agent to dynamically adapt to non-stationary human heuristics, adjusting its policy thresholds based on observed opponent betting styles.